Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3726302.3730117

RESEARCH-ARTICLE

X-Cross: Dynamic Integration of Language Models for Cross-Domain Sequential Recommendation

GUY HADAD, Ben-Gurion University of the Negev, Beer Sheba, Southern District, Israel

HAGGAI ROITMAN, Ben-Gurion University of the Negev, Beer Sheba, Southern District, Israel

YOTAM ESHEL, eBay, Inc., San Jose, CA, United States

BRACHA SHAPIRA, Ben-Gurion University of the Negev, Beer Sheba, Southern District, Israel

LIOR ROKACH, Ben-Gurion University of the Negev, Beer Sheba, Southern District, Israel

Open Access Support provided by:

Ben-Gurion University of the Negev

eBay, Inc.

PDF Download
3726302.3730117.pdf
16 December 2025
Total Citations: 0
Total Downloads: 1247

Published: 13 July 2025

Citation in BibTeX format

SIGIR '25: The 48th International ACM SIGIR Conference on Research and Development in Information Retrieval
July 13 - 18, 2025
Padua, Italy

Conference Sponsors:
SIGIR

SIGIR '25: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (July 2025)
https://doi.org/10.1145/3726302.3730117
ISBN: 9798400715921

X-Cross: Dynamic Integration of Language Models for Cross-Domain Sequential Recommendation

Guy Hadad*
guy_hadad@hotmail.com
Ben-Gurion University of the Negev
Beer Sheva, Israel

Haggai Roitman†
haggair@gmail.com
Ben-Gurion University of the Negev
Beer Sheva, Israel

Yotam Eshel
yeshel@ebay.com
eBay
Netanya, Israel

Bracha Shapira
bracha.shapira@gmail.com
Ben-Gurion University of the Negev
Beer Sheva, Israel

Lior Rokach
liorrk@bgu.ac.il
Ben-Gurion University of the Negev
Beer Sheva, Israel

Abstract

As new products are emerging daily, recommendation systems are required to quickly adapt to possible new domains without needing extensive retraining. This work presents "X-Cross" – a novel cross-domain sequential-recommendation model that recommends products in new domains by integrating several domain-specific language models; each model is fine-tuned with low-rank adapters (LoRA). Given a recommendation prompt, operating layer by layer, X-Cross dynamically refines the representation of each source language model by integrating knowledge from all other models. These refined representations are propagated from one layer to the next, leveraging the activations from each domain adapter to ensure domain-specific nuances are preserved while enabling adaptability across domains. Using Amazon datasets for sequential recommendation, X-Cross achieves performance comparable to a model that is fine-tuned with LoRA, while using only 25% of the additional parameters. In cross-domain tasks, such as adapting from `Toys` domain to `Tools`, `Electronics` or `Sports`, X-Cross demonstrates robust performance, while requiring about 50%-75% less fine-tuning data than LoRA to make fine-tuning effective. Furthermore, X-Cross achieves significant improvement in accuracy over alternative cross-domain baselines. Overall, X-Cross enables scalable and adaptive cross-domain recommendations, reducing computational overhead and providing an efficient solution for data-constrained environments.

CCS Concepts

• **Information systems** → **Recommender systems**; *Cross-domain recommendation*; • **Computing methodologies** → *Natural language processing*; *Language models*; Transfer learning.

*Corresponding author
†Haggai Roitman was still affiliated with eBay during this work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGIR '25, July 13–18, 2025, Padua, Italy
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1592-1/2025/07
https://doi.org/10.1145/3726302.3730117

Keywords

Cross-domain recommendation, language models, natural language processing, dynamic integration, LoRA, parameter and data efficiency

ACM Reference Format:
Guy Hadad, Haggai Roitman, Yotam Eshel, Bracha Shapira, and Lior Rokach. 2025. X-Cross: Dynamic Integration of Language Models for Cross-Domain Sequential Recommendation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25), July 13–18, 2025, Padua, Italy.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3726302.3730117

1 Introduction

The increasing variety of products and services available across online platforms, along with the rapid emergence of new domains, underscores the growing need for sequential recommendation models with fast and efficient cross-domain adaptation. Traditional sequential recommendation models, often tailored to specific content areas, struggle to generalize effectively as new domains emerge at an unprecedented pace. As a motivating example, online marketplaces frequently introduce new and particular product categories, such as "vintage collectibles", "sustainable fashion", "personalized DIY kits" or "specialized home automation devices". These emerging sub-domains challenge conventional models trained on broader categories like "consumer electronics" or "home goods", since it requires nuanced understanding to capture users' preferences in such niche areas. As large language models (LLMs) continue to develop in scope and expressive power, they offer exciting potential for cross-domain applications by leveraging their vast world knowledge to quickly adapt to new domains and tasks [57]. This capacity to operate across numerous and increasingly specific domains positions LLMs as powerful tools for recommendation tasks.

However, effective cross-domain recommendation with LLMs presents significant challenges. The large size and complexity of state-of-the-art language models make them resource-intensive to train even with parameter-efficient fine-tuning (PEFT) techniques. While PEFT, such as Low-Rank Adaptation (LoRA) [22], has reduced the need for extensive retraining, the parameter requirements for adapting large models across multiple domains remains high, often surpassing the computational budgets of many real-world applications. A key challenge is ensuring that cross-domain recommendation tasks capture domain-specific granularity while

remaining adaptable across domains. Current methods [3, 47] rely heavily on the activations of pre-trained weights, which may lack the capacity to generate sufficiently distinctive representations for relatively similar domains. Consequently, these approaches might not adequately capture the unique nuances of each domain.

The need for adaptability in cross-domain recommendation is intensified by the data demands of LLMs. These models typically require large, domain-specific datasets to achieve optimal performance, which is not always available – especially in newly emerging domains where data is scarce or when domain-nuanced recommendations are required. To the best of our knowledge, such an approach, which introduces finer-grained adaptability across the model, has not yet been explored in existing research.

The goal of our work is, therefore, to leverage language models that were trained with user-item interaction histories observed in other (source) domains for sequential recommendation in a new (target) domain. Hence, we wish to learn to **transfer knowledge** that was acquired in source domains to the new target domain; this, while using as **minimum** as possible observed data from the target domain for training. Moreover, we **do not assume knowledge sharing** of users or items between domains.

Trying to address the aforementioned challenges, we introduce the "X-Cross" model – a novel dynamic integration model for cross-domain adaptability that learns to transfer knowledge from several source domain language-models to a new target domain. Given a set of two or more LoRA fine-tuned source domain language models, for each (recommendation prompt) input, X-Cross operates layer-by-layer and computes weights, integrating the strengths of multiple domain-specific models into refined representations. These refined representations are propagated from one layer to another, ensuring domain-specific nuances are preserved while enabling cross-domain adaptability. By leveraging activations from each source domain (LoRA) adapter, the dynamic integration mechanism creates representations that are both granular and adaptable, offering a practical solution.

Such a layer-wise dynamic integration eliminates the need to retrain or modify the original source domain LoRA adapters, allowing our model to achieve performance comparable to newly trained LoRA adapters using just 25% of the parameters required by LoRA. Moreover, X-Cross achieves a competitive and sometime even better recommendation quality to a model that is fine-tuned with LoRA while using 50-75% less training data. We further demonstrate that X-Cross performs better than other alternative language-model based cross-domain recommenders, including alternatives that utilize state-of-the-art mixture-of-LoRA methods.

Overall, our work contributes to developing a parameter-efficient, data-friendly, and adaptable solution for cross-domain sequential recommendation, offering a scalable pathway for recommendation systems in data-limited and rapidly evolving domains.

2 Related Work

We categorize previous related works into three areas of research: language models for sequential recommendation, cross-domain recommendation, and methods employing mixture-of-LoRAs for efficient model fine-tuning. Below, we briefly review these works and highlight their limitations in comparison to our approach.

2.1 Language Models for Sequential-Recommendation

The sequential recommendation task has been extensively studied [2], with many previous works commonly model user-item interaction sequences using Transformer-models; having sequence items represented either by their identities (IDs) (e.g., SASRec [24], BERT4Rec [41], SSE-PT [45]) or by their attributes (e.g., FDSA [56], Trans2D [40]). The rise of large language models (LLMs) in recent years has revolutionized natural language processing (NLP), and specifically their applications to recommendation systems [30, 33, 46] (LLM4Rec). For sequential recommendation tasks, language models have been utilized so far in two primary ways. Firstly, language models have been utilized for feature encoding and augmentation, providing rich context for downstream sequential recommendation pipelines [30]. Secondly, previous works have utilized the in-context learning and instruction following capabilities of LLMs to provide recommendations directly to users. To this end, given user history described in a textual form (prompt), language models were instructed either to select the next item from a list of candidates (e.g., P5 [14], InstructRec [54], RecRanker [36]) or generate a template that represents such an item (e.g., LlamaRec [52], RecPrompt [31]) which is then used to score and rank actual items. In this work, we also utilize a language model for sequential-recommendation, where we cast the task as a multi-choice problem, converting its outputs into scores for ranking candidates for user's next item.

2.2 Cross-Domain Recommendation

The cross-domain recommendation task aims to transfer knowledge between domains to improve performance, particularly in scenarios where the target domain suffers from limited or sparse training data [53]. Language models hold significant potential for such tasks due to their vast world knowledge and advanced reasoning capabilities, which are essential for the generalization required in cross-domain recommendation settings [4, 42].

Among the most relevant works to ours, ZESRec [7] has applied BERT [6] to create semantic representations for zero-shot cross-domain recommendations. TransRec [12] has explored adapter-tuning for transferable recommendations. UniSRec [21] has addressed cross-domain sequential recommendation by utilizing a Mixture-of-Experts module to integrate the BERT representations into the recommendation task. VQ-Rec [19] has adapted textual embeddings generated by pre-trained language models by leveraging vector-quantization. RecFormer [27] has utilized language representations to model user preferences and item features, enabling effective next-item prediction, particularly in low-resource and cold-start scenarios.

Compared to our work, existing models rely on fixed representations derived from the language model [19, 21], which limits their ability to fully leverage the potential of the model during fine-tuning for the source domain. This is due to the inherent constraints of pre-trained language models when applied to recommendation tasks [25]. In addition, utilizing only the final hidden state disregards the rich knowledge embedded within intermediate layers [43], which may contain valuable information relevant to cross-domain recommendation tasks.

2.3 Mixtures of LoRAs

With the rise of parameter-efficient fine-tuning (PEFT) methods [8, 13], driven by the increasing scale of models and datasets, LoRA [22] and its variants [5, 23, 34] have emerged as a popular approach for fine-tuning not only large language models but also other large-scale neural networks across diverse applications [32, 55]. By significantly reducing the number of trainable parameters, LoRA enables efficient adaptation without modifying the pre-trained model weights, making fine-tuning more computationally feasible. The integration of several models has gained a significant traction in various generative AI applications [48, 49]; with one of the most popular approaches being the integration of several LoRA adapters, each fine-tuned on a different task [11, 38]. Previous works have explored various strategies for integrating adapters in machine-learning models. Some works focus on integrating adapters exclusively during the inference phase to enhance model performance without additional training overhead [38]. Others extend this approach by integrating adapters during training, often incorporating Mixture-of-Experts [9, 10, 59] (MoE) techniques to better optimize the models for specific tasks [3, 47]. Furthermore, integration mechanisms have been proposed, with most approaches relying on pre-trained model representations as the foundation for the integration process. While some methods require domain labels during the training process [11, 47], they still often face challenges in cross-domain tasks where true labels may be ambiguous or unavailable [47]. These approaches usually assume distinct and well-separated domains during training, which is rarely the case in real-world recommendation tasks where domain boundaries can be vague. Moreover, these methods can struggle when the pre-trained model representations are not sufficiently distinctive, as is often the case in recommendation tasks [25].

3 Recommendation Framework

In this section we describe our proposed solution for cross-domain sequential recommendation. We begin by providing the basic background required to understand the cross-domain sequential recommendation task and how we leverage a language-model to solve it. We then present the X-Cross model which integrates several source domain language-models to recommend items in a new target domain. We describe its four different stages and how to utilize it for cross-domain sequential recommendation.

3.1 Background

We provide the necessary background for our model. First, we define the cross-domain sequential recommendation task. Next, we explain how we frame it as a multiple-choice problem using a language model. We then outline the model training process and introduce LoRA for domain-specific fine-tuning.

3.1.1 Cross-Domain Sequential Recommendation Task. Let U represent the set of users, I the set of items, and $S_u = (i_1, i_2, \ldots, i_N)$ represent the (sequence) history of N last items from I that user $u \in U$ has interacted with. Given user history S_u, the sequential recommendation task is to predict the next user interaction $i_{N+1} \in I$.

In the cross-domain recommendation setting that we study in this work, items in I are assumed to belong to a new or scarcely represented target domain D_{target}; where we aim to leverage recommendation models that were trained using user-item interaction histories that were observed in other source domains $\mathcal{D}_{source} = \{D_1, D_2, \ldots, D_n\}$. Hence, we wish to learn to **transfer knowledge** that was acquired in the source domains to the new target domain. Moreover, we wish to achieve such transferability assuming **minimum** observed data from the target domain.

Sequential Recommendation Prompt

History:
1: SE 8787MS-SP 9-in-1 Emergency Shovel Tool Kit.
2: SE CCH7-1G 7-IN-1 Green Survival Whistle.
3: Attmu 2 Pack Outdoor Survival Paracord Bracelet.
4: Garmin DeLorme Atlas & Gazetteer Paper Maps - Arizona.
5: DANTENG Phone Cable Black
Candidate:
Mylar Men's Emergency Thermal Blankets (10 Pack)

Figure 1: Example prompt for sequential recommendation task having a user history with 5 items and a candidate item.

3.1.2 Sequential Recommendation as a Multiple-Choice Problem. In this work, we use a language model to recommend the next item to a user $u \in U$, given user's interaction history S_u. To this end, we first cast the recommendation task as a multiple-choice problem. For any given candidate next item $i \in I$, the input to the language model is simply expressed in a textual form (prompt) as follows:

$$\text{Prompt} = [\text{History: } S_u, \text{Candidate: } i]$$

Figure 1 illustrates an example prompt with a history that contains five items and a candidate item.

We now define $score(S_u, i)$ as the model "confidence" that item i's text is likely to follow the textual representation of the user history S_u in the prompt. Using the language-model as an **encoder**, in this work, we obtain such a score by simply pooling the representation of the last hidden layer of the language model and apply a simple scoring "head" over it (see details in Section 3.2.5). Given a set of candidate items in I, using such a prompting (and scoring) approach, allows us to score any candidate item and select the next item as the one with the highest score.

3.1.3 Model Training. Following a common methodology in training recommender-systems [39, 50, 51], we train the (language) model by sampling several negative items I_{neg} for each true item i with which a given user u has interacted. For each true candidate item $i \in I$, the model is, therefore, trained to maximize $score(S_u, i)$ using the negative log-likelihood loss:

$$\mathcal{L}_i = -\log\left(\frac{\exp(score(S_u, i))}{\sum_{i' \in \{i\} \cup I_{neg}} \exp(score(S_u, i'))}\right). \quad (1)$$

3.1.4 LoRA for Domain Specific Fine-Tuning. Low-Rank Adaptation (LoRA) [22] is a method designed to efficiently fine-tune large pre-trained models. To this end, LoRA introduces additional trainable low-rank matrices into specific layers, based on the hypothesis that the updates required for fine-tuning reside in a low-dimensional

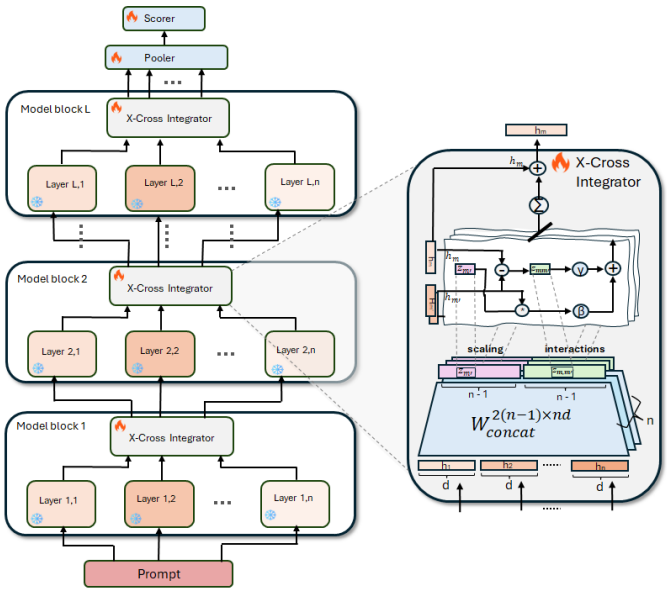

Figure 2: X-Cross model architecture. Each source domain language model is implemented with several Transformer (vertical) layers. On the left side: at each layer, the "hot"-trainable integrator receives activations from the "frozen" layers and then passes the integrated representations to the next layer. On the right side: a "zoom-in" into an X-Cross integrator located at one of the network layers.

subspace. Therefore, instead of updating all the parameters of the pre-trained model, LoRA freezes the original model weights and introduces a learnable adjustment:

$$\mathbf{W}_{\text{LoRA}} = \mathbf{W} + \alpha \cdot \mathbf{AB}, \tag{2}$$

where:

- $\mathbf{W} \in \mathbb{R}^{d\times k}$ represents the frozen pre-trained weights, with d denoting the input dimension and k the output dimension.
- $\mathbf{A} \in \mathbb{R}^{d\times r}$ and $\mathbf{B} \in \mathbb{R}^{r\times k}$ are trainable low-rank matrices, where $r \ll \min(d, k)$, representing the adaptation rank.
- $\alpha \in \mathbb{R}$ controls the adaptation magnitude.

In this work, for each source domain $D \in \mathcal{D}_{source}$, we train a unique LoRA adapter to capture domain-specific patterns.

3.2 The X-Cross Model

The X-Cross model (hereinafter termed "X-Cross" for short) learns to recommend items in a new target domain by integrating multiple pre-trained language models, with each model that was previously adapted with LoRA for a specific source domain. X-Cross dynamically combines activations from the LoRA-enhanced "encoders" across all source domains, allowing the model to leverage domain-specific knowledge while adapting flexibly to the input context.

Figure 2 now illustrates the architecture of X-Cross. As a preliminary step, we assume the availability of $n \geq 2$ source domain (language) models, assuming each model is fine-tuned with its own dedicated LoRA adapter. The source models are visually represented in a vertical arrangement within the figure. X-Cross pools information from all source domain models by introducing a dedicated integrator at each (network) layer, enabling the computation of scaling factors (weights) dynamically for each source domain. This per-layer integration design captures the varying importance of layers in recommendation tasks, where different layers contribute uniquely to relevance signals. By dynamically adjusting the scaling factors at each layer, X-Cross ensures that the integrated representation effectively balances shared and domain-specific knowledge. Our experimental results (see Section 4.5) further validate the necessity of per-layer integration, demonstrating its critical role in achieving effective cross-domain recommendations. Overall, X-Cross implementation includes four main stages. The first three stages are applied at each layer-level for each source domain, while the final stage is applied on the outputs of the last layers of all source domains. For simplicity of presentation, the detailed stages describe representation calculations that are performed over every sequence input token (i.e., we exclude the sequence-dimension).

3.2.1 Stage 1: Concatenation of source domain representations. For each source domain D_m ($1 \leq m \leq n$), let $\mathbf{h}_m^{(l)}$ denote the output of the LoRA-enhanced encoder for that domain at layer l ($1 \leq l \leq L$) of the network, which is obtained as follows:

$$\mathbf{h}_m^{(l)} = \left[\mathbf{W}_m^{(l)} + \mathbf{A}_m^{(l)}\mathbf{B}_m^{(l)}\right] \cdot \mathbf{x}_m^{(l)}, \tag{3}$$

where $\mathbf{W}_m^{(l)} \in \mathbb{R}^{d\times k}$ denotes the **frozen** pre-trained weights matrix of the m-th encoder at layer l; $\mathbf{A}_m^{(l)} \in \mathbb{R}^{d\times r}$ and $\mathbf{B}_m^{(l)} \in \mathbb{R}^{r\times k}$ are the LoRA weights specific to the encoder; and $\mathbf{x}_m^{(l)} \in \mathbb{R}^k$ represents the input to layer l. Here we note that, we assume that the fine-tuned LoRA weights (i.e., $\mathbf{A}_m^{(l)}$ and $\mathbf{B}_m^{(l)}$) are **frozen** as well.

In contrast to previous models [3, 47] that have applied integration directly on activations from pre-trained weights (i.e., $\mathbf{Wx}$), our approach routes the adapted activations (i.e., $(\mathbf{W} + \mathbf{AB})\mathbf{x}$) produced after domain-specific LoRA adapters. This adjustment addresses a known limitation in recommendation tasks, where pre-trained model activations often lack sufficient knowledge about recommendation-specific data [25]. By leveraging the enriched activations from LoRA adapters, X-Cross dynamically adjusts weights across representations from different domains without requiring explicit supervision for the integrator. Instead, the weights are computed adaptively for each input, guided solely by the supervision provided by the label of the sequential recommendation task in the target domain. This approach enables label-free integration of domain representations, enhancing flexibility and performance in cross-domain recommendations.

At this stage we obtain the concatenated representation across all n encoders:

$$\mathbf{h}_{\text{concat}}^{(l)} = [\![\mathbf{h}_1^{(l)}; \mathbf{h}_2^{(l)}; \ldots; \mathbf{h}_n^{(l)}]\!],$$

where $[\![\cdot;\cdot]\!]$ represents the concatenation operation along the feature dimension. Each $\mathbf{h}_m^{(l)} \in \mathbb{R}^d$ contributes to the final representation, resulting in $\mathbf{h}_{\text{concat}}^{(l)} \in \mathbb{R}^{n\cdot d}$.

3.2.2 Stage 2: Dynamic scaling. At this stage, the concatenated representation $\mathbf{h}_{\text{concat}}^{(l)}$ undergoes a trainable linear transformation to compute the weights for domain-specific layer representations.

Formally, the weights are computed as:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}_{\text{concat}} \cdot \mathbf{h}^{(l)}_{\text{concat}}, \tag{4}$$

where $\mathbf{W}^{(l)}_{\text{concat}} \in \mathbb{R}^{2n(n-1)\times(n\cdot d)}$ is a trainable weight matrix. Therefore, the output $\mathbf{z}^{(l)} \in \mathbb{R}^{2n(n-1)}$ contains for each domain: 1) n-1 **domain-specific** scaling factors which modulate the contribution of other domains to the integrated representation for that domain; 2) n-1 **interaction terms** per each **pair** of that domain with another domain. Hence, in total, $\mathbf{z}^{(l)}$ contains $2n(n-1)$ learnable weights. These weights in $\mathbf{z}^{(l)}$ enable the model in the next stage to dynamically balance the contributions from each source domain while capturing complex inter-domain dependencies, resulting in richer and more nuanced integrated representations.

Unlike conventional methods that rely on softmax to produce only positive scaling factors [47, 59], our approach supports both positive and negative scaling. This flexibility allows X-Cross to **suppress irrelevant domain contributions**. This enhances X-Cross's ability to focus on meaningful features for the recommendation task. By allowing negative scaling, X-Cross can effectively downscale or even "move away" from less relevant domains when "constructing" the integrated representation. This capability is particularly advantageous in scenarios where certain domain adapters introduce noise or irrelevant information, ensuring the final representation prioritizes the most relevant features.

3.2.3 Stage 3: Representation Refinement and Integration: This stage ensures that the integrated representation of each domain D_m (i.e., $\mathbf{h}^{(l)}_m$) would "blend" knowledge from all domains while dynamically adapting to the context of each sample. The weights $\mathbf{z}^{(l)}$ modulate both the direct contributions of domain-specific outputs and their interactions with other domains. We next detail how these weights are applied to construct the final representation of each domain.

For each domain-specific representation $\mathbf{h}^{(l)}_m$, we compute an integrated refined representation, denoted as $\tilde{\mathbf{h}}^{(l)}_m$. This representation incorporates contributions from other domains while excluding the current domain D_m, and is calculated as follows:

$$\tilde{\mathbf{h}}^{(l)}_m = \mathbf{h}^{(l)}_m + \sum_{m'\neq m} \left(\beta \cdot \mathbf{z}^{(l,m)}_{[m']} \cdot \mathbf{h}^{(l)}_{m'} + \gamma \cdot \mathbf{z}^{(l,m)}_{[m,m']} \cdot \left(\mathbf{h}^{(l)}_m - \mathbf{h}^{(l)}_{m'}\right)\right) \tag{5}$$

Here, the term $\beta \cdot \mathbf{z}^{(l,m)}_{[m']} \cdot \mathbf{h}^{(l)}_{m'}$ integrates the **direct influence** of other domains $D_{m'}$ into the refined representation of domain D_m, where $\mathbf{z}^{(l,m)}_{[m']}$ is the corresponding scaling factor for domain $D_{m'}$. The term $\gamma \cdot \mathbf{z}^{(l,m)}_{[m,m']} \cdot \left(\mathbf{h}^{(l)}_m - \mathbf{h}^{(l)}_{m'}\right)$ further captures the **inter-domain interactions** by measuring the differences between domain representations, enriching the unified representation for the cross-domain task. The term $\mathbf{z}^{(l,m)}_{[m,m']}$ further denotes the relative weight of the inter-domain interaction between the two domains.

We note that, the summation in Eq 5 excludes domain D_m itself, ensuring that the refined representation focuses on its interactions with the other $n-1$ domains. The scalars β and γ are hyperparameters that control the contribution strength of the two terms.

The residual connection allows each domain-specific representation to retain its original characteristics by avoiding incorporation of superfluous knowledge, reducing noise and preserving key domain knowledge.

The adaptive integration mechanism operates iteratively at each layer $1 \leq l \leq L$, enabling the model to dynamically balance shared and domain-specific knowledge. Through this progressive refinement, the model captures complex inter-domain relationships, producing robust integrated representations.

3.2.4 Stage 4: Final weighted summation: After processing through all layers, the refined domain-specific representations are aggregated into a single integrated representation. This is achieved through a weighted summation of the final layer outputs from all source domain encoders:

$$\mathbf{h}^{\text{final}} = \sum_{m=1}^{n} w_m \cdot \tilde{\mathbf{h}}^{(L)}_m, \tag{6}$$

where $\tilde{\mathbf{h}}^{(L)}_m$ is the output of the m-th encoder at the last layer L, and $w_m \in \mathbb{R}$ is the learnable weight of domain m.

3.2.5 Candidate scoring. Once X-Cross has encoded the input prompt, we utilize the integrated representation $\mathbf{h}^{\text{final}}$ ($\in \mathbb{R}^d$) to estimate the "likelihood" that the prompt's candidate item is the next item given user's interactions history (see again Section 3.1.2). To this end, $\mathbf{h}^{\text{final}}$ undergoes a pooling operation to extract a compact vector that "summarizes" the sequence information. Specifically, we employ a *contextual token pooling* mechanism that focuses on the representation of the first ([CLS]) token in the sequence. This pooling method is consistent with the one proposed in the DeBERTa model [16], which is the language model we utilize in our experiments (see more details in Section 4.1).

Formally, the pooled representation $\mathbf{h}_{\text{pooled}}$ is defined as follows:

$$\mathbf{h}_{\text{pooled}} = \text{GELU}\left(\mathbf{W}_p \cdot \mathbf{h}^{\text{final}}_{\texttt{[CLS]}} + \mathbf{b}_p\right), \tag{7}$$

where:

- $\mathbf{h}^{\text{final}}_{\texttt{[CLS]}} \in \mathbb{R}^d$ represents the hidden state corresponding to the first token ([CLS] context token).
- $\mathbf{W}_p \in \mathbb{R}^{d\times d}$ and $\mathbf{b}_p \in \mathbb{R}^d$ are learnable parameters.
- GELU$(\cdot)$ is the Gaussian Error Linear Unit activation function [18], as used in the DeBERTa model [16].

Finally, we score the prompt's candidate item i as the next item in the user's sequence by applying a simple scoring (regression) "head" over the pooled representation $\mathbf{h}_{\text{pooled}}$. For that, we implement the "scorer" using a simple linear layer, calculated as follows:

$$score(S_u, i) = \mathbf{V}_c^T \cdot \mathbf{h}_{\text{pooled}} + b_c, \tag{8}$$

where $\mathbf{V}_c \in \mathbb{R}^d$ and $b_c \in \mathbb{R}$ are the learnable weights vector and bias term of the linear layer, respectively.

To remind, $score(S_u, i)$ represents the model's predicted score for the given prompt's candidate item i and user history S_u (we kindly refer the reader again to Section 3.1.3 for model training details). Further to remind, at training time, we train the model to score the true candidate i along with a sample of negative items I_{neg}. At inference time, given a recall-set of candidate items, we simply choose the next user item as the one with the highest score.

4 Evaluation

In this section we evaluate X-Cross. We first outline the experimental setup (Section 4.1). We then evaluate X-Cross over different domains and baselines, including traditional baselines, cross-domain baselines, and alternative integration strategies (Section 4.2). Our evaluation further emphasizes the competitive performance of X-Cross to training a new LoRA adapter. Next, we assess X-Cross efficiency by examining its performance under limited training data conditions (Section 4.3) and the effect of the number of layers in gradient calculations (Section 4.4). We then conduct an ablation study (Section 4.5) and conclude with an analysis of the factors driving the diverse convergence behaviors observed across datasets (Section 4.6).

4.1 Experimental Setup

4.1.1 Datasets. We curate four datasets from the Amazon reviews corpus [37], focusing on the domains of `Electronics`, `Sports`, `Tools` and `Toys`, which are commonly used for sequential recommendation tasks [24, 28, 41]. Each dataset complies with the widely adopted "core 5" criteria [17, 39, 44], ensuring that every user and item has at least five interactions. Further following [1], to accommodate the input constraints of the language model in our evaluation, we limit each user's interaction history to a minimum of 5 and a maximum of 15 unique items. We further represent each item within the input prompt by its title (see again Figure 1).

We construct user histories by randomly selecting one interaction per day from each user's activity. This approach addresses a significant limitation of the Amazon datasets (2018), which only provide timestamps at the **day level**, leading to potential inaccuracies in time-sensitive tasks like sequential recommendation [20]. By ensuring a consistent chronological sequence, this approach helps to preserve temporal order across days, reducing ambiguity and improving the reliability of sequence modeling.[1]

Table 1: Dataset statistics. For each domain, per number of interactions we also report the data density. Except for `Tools` dataset, all other domain dataset numbers are reported after user sampling is applied.

Target Domain	# Users	# Items	# Interactions	Source Domains
Electronics	26,319	12,031	165,541 (5.2e-4)	Tools+Toys
Sports	19,244	10,036	120,122 (6.2e-4)	Toys+Electronics
Tools	10,962	5,820	66,164 (1.0e-3)	Sports+Electronics
Toys	21,342	10,457	131,485 (5.9e-4)	Sports+Electronics

As shown in Table 1, our datasets exhibit low density, which poses a significant challenge for traditional recommendation models. However, such data sparsity allows us to test our core hypothesis, that language models that are trained in different domains can still be used to provide recommendations in other, even different, domains. To balance computational efficiency with comprehensive evaluation, except for `Tools` domain, which is a relatively small dataset, we focus on a subset of users, selecting 40% of users from the `Toys` and `Sports` domains and 30% from the `Electronics` domain, ensuring diverse and manageable datasets for analysis.

4.1.2 Implementation. To recall, we predict the next item using a **multiple-choice** task (see again Section 3.1.2). Accordingly, during both training and inference, for each next true item i, we sample 29 negative items (resulting with 30 choices overall for the model to "choose" from). Following [29, 41], we obtain negative samples using a popularity-based sampling strategy. During inference, we predict the next item (out of 30) as the one with the highest score.

As the backbone language model in our evaluation, we adopt the DeBERTa V3 base model [15, 16]. We make this language model choice primarily to ensure a **fair comparison** with other cross-domain baseline models [19, 21, 27], which have used similar model capacities (e.g., BERT) in their evaluation. Additionally, we choose DeBERTa model as our base (language-model) encoder since it was also evaluated in the original LoRA paper [22]. Therefore, such a choice provides a consistent and robust benchmark for assessing the effectiveness of X-Cross in cross-domain recommendation tasks. To comply with DeBERTa's maximum sequence length (512 tokens), we truncate each item's title to a maximum of 8 words.

We implement **X-Cross** using PyTorch and train the model for 40 epochs on two NVIDIA V100 GPUs, each with 32 GB of memory. The architecture utilizes $n = 2$ source domains ("experts"), where we use a holdout-set to select the top-two source domains with best zero-shot performance on the target domain. To enhance the model's efficiency and specialization, we only modify the top-9 layers of the source domain models. This decision is based on the understanding that the top layers of language models typically capture more abstract and high-level knowledge [43], making them better suited for cross-domain adaptation. Table 1 further details the source domains that we consider per each target domain.

We fine-tune the model using AdamW optimizer [35], configured with a learning rate of 5×10^{-5}, a weight decay of 0.01, and a batch size of 1 to address GPU memory constraints[2]. We configure training hyperparameters using a holdout set, setting $\beta = 0.5$ and $\gamma = 0.4$ for optimal performance. Additionally, for LoRA fine-tuning, we employ a rank $r = 16$ and a scaling factor $\alpha = 32$, ensuring efficient parameter utilization while maintaining strong task performance [22].

4.1.3 Baselines. We compare X-Cross performance to several types of baselines. We first evaluate several state-of-the-art "traditional" single-domain sequential recommendation models, namely: **SASRec** [24] – a widely adopted self-attentive (causal) sequential recommendation model; **BERT4Rec** [41] – a bidirectional transformer-based model which uses a masked learning approach; **FDSA** [56], which leverages self-attentive blocks to model item and feature transition patterns; **S^3-Rec** [58], which pre-trains the model to maximize mutual information for enhanced feature fusion; and lastly, **LLM-Rec** [42], a language model that represents both user history and the next item within a shared embedding space. While this model is generally designed for multi-domain settings, here we apply it to a single-domain setting due to the **lack of overlapping users across domains**. For this evaluation, we specifically use an

[1]We recognize that such an approach does not account for potential behavioral dependencies between multiple interactions within the same day, which could provide additional insights into user preferences.

[2]We note that, each batch basically contains 30 prompts used to implement the model's multiple-choice task.

encoder-only language model because it achieves the best results. Additionally, to ensure a fair comparison, we use the same encoder architecture (DeBERTa) as in our approach.

As a first-line of baselines for cross-domain recommendation, we fine-tune a domain-specific LoRA model [22] for each **source** domain. Here we fine-tune each model independently on its respective (source) domain and evaluate it in a **zero-shot** setting on other domains. These set of baselines allows to assess the potential of transferability among different domains.

We next evaluate three state-of-the-art cross-domain sequential-recommendation models, namely: **UniSRec**, **VQ-Rec** and **RecFormer**. All these models require pretraining. For fair comparison, we pretrain these models on the exact source domains that we use for **X-Cross** and then fine-tune them on the target domain. **UniSRec** [21] equips textual item representations with a mixture-of-experts (MoE)-enhanced adapter for domain fusion and adaptation; leveraging item-sequence and sequence-sequence contrastive learning tasks to pre-train transferable sequence representations. **VQ-Rec** [19] learns vector-quantized item representations for transferable sequential recommendation [19], enabling effective representations across domains. Finally, **RecFormer** [27] leverages language representations to model user preferences and item features, enabling effective next-item prediction, particularly in low-resource and cold-start scenarios.

As two competitive alternative baselines that also focus on integrating domain-specific knowledge, we consider **XLoRA** [3] and **MeteoRA** [47]. The **XLoRA** baseline incorporates core techniques from the XLoRA framework [3] into our setting. Specifically, we use embeddings from the last layer of the pre-trained model to scale all layer "experts", leveraging cross-layer attention to improve generalization across domains. Further adapting the **MeteoRA** framework [47], we implement a gating network for each layer. These networks dynamically integrate adapters by using embeddings from the pre-trained model as inputs, ensuring a more effective integration of domain-specific knowledge. Finally, we implement **Pooler + Scorer** baseline, which simplifies the architecture by fine-tuning only the pooler and scorer layers of the best-performing pre-trained model. We use this baseline to examine the necessity of integrating multiple domain-specific components.

4.1.4 Evaluation Metrics. Following previous works [39, 50, 51], we divide each dataset into training, validation, and testing sets based on user splits, following a 3:1:1 ratio. We asses model performance using standard metrics for top-k recommendation tasks. Specifically, we calculate Hit@k for $k = \{1, 3, 10\}$, and MRR for $k = 10$. Following [24, 26, 58], we conduct negative sampling during the evaluation phase in a manner consistent with the training process, ensuring reliable and comparable evaluation results. Finally, statistical significance of X-Cross performance is evaluated throughout using a two-tailed paired Student's t-test ($p \leq 0.05$).

4.2 Overall Performance

We analyze the overall performance of **X-Cross** compared to the different types of baselines. First, examining the result of single-domain baselines, the **LLM-Rec** baseline demonstrates a significant performance improvement over the others. This showcase the benefit of leveraging embeddings derived from all item titles in the user's history. This holistic representation of user history enables **LLM-Rec** to outperform models that treat items' text independently.

Next, as we can observe, those baselines that we fine-tune with LoRA using a multiple-choice task on the target domain, perform even better than **LLM-Rec**. Furthermore, the cross-domain potential becomes evident, as the baselines that we evaluate using a zero-shot setting also outperform most single-domain baselines.

Next, except for **LLM-Rec**, the cross-domain baselines achieve significantly better results than their single-domain counterparts (with **RecFormer** as the top performer). While these cross-domain baselines may not match LoRA fine-tuning's results for single-domain tasks, this does not necessarily reflect poorly on their cross-domain capabilities. Instead, it highlights the strength of holistic user history representation, as seen in both **LLM-Rec** and LoRA fine-tuning and also the potential of cross-domain recommendation among single-domain baselines.

We next examine the performance of the domain integration baselines, where we observe a mixed performance both among themselves and compared to other baselines. Specifically, in some cases, these baselines perform worse than the **Pooler + Scorer** baseline, which fine-tunes the best source domain's pooler and scorer. However, these integration approaches still provide a strong baseline, particularly when considering their efficiency.

Finally, we first compare **X-Cross** side-by-side with those baselines that require a new LoRA adapter. Even though **X-Cross** requires significantly fewer parameters compared to training a new LoRA adapter, on `Sports` and `Toys` domains it almost reach the same accuracy of these baselines, while for `Tools` and `Electronics` domains it even outperforms these baselines. Furthermore, compared to alternative integrator-baselines, **X-Cross** exceed their performance by a large margin. All in all, these results serve as a strong evidence that **X-Cross** serves as both efficient and effective solution for cross-domain sequential recommendation tasks.

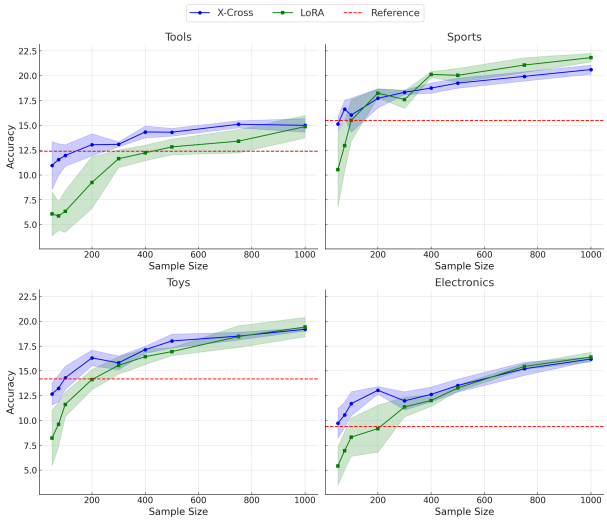

Figure 3: Accuracy (Hit@1) comparison across datasets for X-Cross and LoRA. The dashed red-line denotes the performance of the reference model.

Table 2: Overall performance of X-Cross and baselines. Bold values denote the best performer. Superscripts s and i and subscript c denote a (statistical) significant difference ($p \leq 0.05$) with the best single-domain, integrators and the best cross-domain baselines, respectively.

Model	Tools				Sports				Toys				Electronics			
	Hit@1	Hit@3	Hit@10	MRR	Hit@1	Hit@3	Hit@10	MRR	Hit@1	Hit@3	Hit@10	MRR	Hit@1	Hit@3	Hit@10	MRR
Single Domain Recommendation																
SASRec	6.38	14.23	37.07	13.26	9.79	22.34	51.96	19.99	8.36	17.92	41.46	16.13	6.34	16.19	44.76	15.05
BERT4Rec	4.79	11.13	33.33	11.02	7.85	18.08	45.54	16.56	6.11	14.71	38.86	13.52	5.07	12.96	39.17	12.59
FDSA	5.43	12.59	36.84	12.19	8.21	20.21	49.52	18.02	7.14	16.37	44.25	15.55	6.04	16.00	43.01	14.53
S^3-Rec	5.15	11.67	35.02	11.56	8.68	20.21	50.58	18.46	6.75	16.44	41.51	14.72	5.62	14.67	43.69	13.98
LLM-Rec	14.77	35.70	74.14	30.14	22.29	47.96	82.49	39.38	23.45	49.87	83.91	40.87	25.57	**51.99**	**85.03**	42.82
LoRA Fine-Tuning on Source Domain / Zero-shot on Target Domain																
Tools-LoRA	18.74	39.26	74.87	33.69	13.30	28.11	58.35	24.64	13.91	26.40	53.99	23.79	8.85	20.11	45.55	17.75
Sports-LoRA	12.40	25.58	55.95	22.74	27.98	**51.49**	**82.62**	43.49	14.10	25.49	49.50	22.92	8.49	17.63	44.03	16.48
Toys-LoRA	11.26	22.16	48.34	20.06	14.52	27.54	55.55	24.71	27.99	52.02	84.77	44.05	8.98	18.96	45.21	17.47
Electronics-LoRA	12.40	26.32	50.39	21.59	15.82	30.32	59.08	26.74	14.34	25.84	52.85	23.74	25.56	51.77	84.08	42.40
Cross-domain Recommendation																
UniSRec	7.80	16.96	42.64	15.95	13.07	27.49	58.92	24.39	10.03	19.98	46.10	18.46	7.66	17.21	46.33	16.28
VQ-Rec	6.84	15.55	41.04	14.42	10.81	24.06	55.18	21.58	8.03	17.36	43.15	16.08	7.09	16.36	43.84	15.34
RecFormer	12.80	26.83	54.65	23.35	18.76	36.82	69.15	31.83	16.74	30.31	57.08	26.96	14.19	29.20	60.08	25.59
Domain Integration																
XLoRA	18.01	36.21	69.95	31.44	24.45	44.89	77.01	38.68	23.28	45.07	79.90	38.66	15.14	32.43	66.66	28.23
MeteoRA	17.74	36.53	70.27	31.26	21.38	41.62	72.04	35.10	22.30	42.96	76.76	37.00	17.65	37.10	72.38	31.91
Pooler + Scorer	14.09	31.19	61.70	26.48	22.97	42.56	74.69	36.74	20.73	39.73	73.23	34.45	15.69	33.91	68.18	29.07
X-Cross (ours)	$\mathbf{20.11}^{i,s}_{c}$	$\mathbf{40.36}^{i,s}_{c}$	$\mathbf{76.06}^{i,s}_{c}$	$\mathbf{34.87}^{i,s}_{c}$	$\mathbf{28.40}^{i,s}_{c}$	$50.09^{i,s}_{c}$	82.05^{i}_{c}	$43.36^{i,s}_{c}$	$\mathbf{28.23}^{i,s}_{c}$	$\mathbf{52.66}^{i,s}_{c}$	$\mathbf{85.66}^{i,s}_{c}$	$\mathbf{44.65}^{i,s}_{c}$	$\mathbf{26.25}^{i}_{c}$	50.72^{i}_{c}	84.71^{i}_{c}	$\mathbf{42.84}^{i}_{c}$

Table 3: Models performance under training data limitations.

Target Domain	Significant for < M Samples	X-Cross to Exceed Baseline	LoRA to Exceed Baseline	Gap (%)
Tools	1000	200	500	60.0%
Sports	100	75	200	62.5%
Toys	300	100	300	66.7%
Electronics	300	50	300	83.3%

4.3 Efficiency on Limited Training Data

We next wish to evaluate X-Cross performance under limited training data settings. To this end, for each target domain, we compare X-Cross side-by-side with a model that is fine-tuned with LoRA. Here, we remind that, using $n = 2$ source domains, X-Cross requires only 25% of the parameters of LoRA[3]. Yet, as we shall shortly demonstrate, despite this modest decrease in parameter size, X-Cross achieves competitive results against LoRA in terms of performance, underscoring its effectiveness.

We evaluate both models (X-Cross and LoRA) using training datasets of varying sizes: {50, 75, 100, 200, 300, 400, 500, 750, 1000}. For each training dataset size, we randomly sample five distinct subsets of the specified size from the full dataset. We train both models separately on each of these subsets. As a reference model for minimum performance requirement, for each target domain, we consider the source model with best zero-shot performance which is trained on one of the other (source) domains (e.g., for `Tools`, the reference model is LoRA-fine-tuned either on the `Sports`, `Toys` or `Electronics` domain).

[3]Per each layer l, X-Cross learns the weights matrix $\mathbf{W}^{(l)}_{\text{concat}}$ which for $n = 2$ translates to 4*2*768 total parameters; whereas LoRA learns two weight matrices $\mathbf{A}^{(l)}$ and $\mathbf{B}^{(l)}$, together having 2*16*768 total parameters.

We describe our results in Figure 3, where we evaluate all three models on the full test-set to identify the minimum amount of training data required for either LoRA or X-Cross to surpass the performance of the reference model (further represented by the red dashed line). We further summarize the training data requirements across domains of both models in Table 3. The first column indicates the maximum number of samples (M) in the training set for which X-Cross performance remains statistically significantly better than LoRA. The second and third columns represent the minimum number of training samples required for X-Cross and the standard LoRA adapter to exceed the reference model, respectively. The final column, "Gap (%)", shows the percentage reduction in the amount of training data required by X-Cross compared to LoRA. Further note that, the values in Table 3 represent the **smallest** sampled training dataset sizes at which the performance threshold is exceeded. If the threshold is not surpassed at a given size (e.g., 100) but is at the next increment (e.g., 200), the latter value (200) is recorded in the table to reflect this milestone in the sampling process. Overall, these empirical results indicate that X-Cross attains a steeper initial learning curve, reaching to above-reference performance with substantially fewer samples.

4.4 Impact of Number of Model Layers

We further investigate the contribution of individual layers within X-Cross. To this end, we vary the number of layers integrated into the model. Specifically, we evaluate its performance when scaling and integrating either 1, 2, 4, or 8 layers during the first three stages, starting from the **topmost** layer and down. The results in Figure 4 show that, while adding layers generally improves recommendation accuracy, the performance gains are not uniform. Some layers

Table 4: Ablation study results: Impact of removing key components on performance.

Variant	Tools				Sports				Toys				Electronics			
	Hit@1	Hit@3	Hit@10	MRR	Hit@1	Hit@3	Hit@10	MRR	Hit@1	Hit@3	Hit@10	MRR	Hit@1	Hit@3	Hit@10	MRR
X-Cross	20.11	40.36	76.06	34.87	28.40	50.09	82.05	43.36	28.23	52.66	85.66	44.65	26.25	50.72	84.71	42.84
- Layers	16.60	35.11	68.99	30.20	24.24	44.58	76.57	38.53	22.96	45.26	81.73	38.73	15.67	33.33	69.47	29.15
- Interactions	17.46	38.49	75.01	32.62	26.73	49.55	81.84	42.10	26.10	50.97	83.35	42.51	23.02	46.12	80.72	38.95
- Experts	19.33	39.67	73.78	33.68	27.57	49.75	81.55	42.62	25.81	49.03	83.98	41.80	26.06	50.06	82.88	42.13

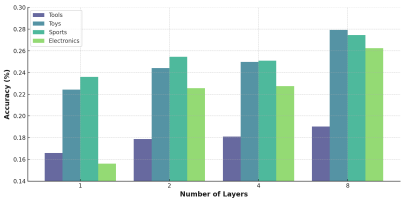

Figure 4: Accuracy (Hit@1) vs number of layers.

capture more domain-relevant knowledge and significantly impact performance, while others contribute less.

4.5 Ablation Study

We next perform an ablation study to evaluate the relative contributions of the key components of X-Cross. For that, each time, we systematically alter the full implementation of X-Cross by removing a specific component and measuring its impact on performance. We summarize the results of the ablation study in Table 4. First, by setting both $\beta = 0$ and $\gamma = 0$ (in Eq. 5) we remove the dynamic integration mechanism between source domains at each layer, allowing each one to operate independently and combining their outputs only at the **final stage** (Eq. 6). As we can observe, this simplification (denoted "**-Layers**" in Table 4) which eliminates layer-wise collaboration, leads to a significant (and largest) performance drop, emphasizing the critical role of dynamic inter-layer integration.

Next, we exclude the interactions component during the integration process by setting $\gamma = 0$ while still keeping $\beta = 0.5$ (denoted "**-Interactions**" in Table 4). This modification, which disables the interaction terms $(\mathbf{h}_m^{(l)} - \mathbf{h}_{m'}^{(l)})$, reduces the model's ability to capture nuanced inter-domain relationships, resulting in a notable performance decline.

Lastly, we eliminate the contributions of other source domains entirely by setting $\beta = 0$ while still keeping $\gamma = 0.4$ (denoted "**-Experts**" in Table 4). This modification leaves the domain-specific model and the interactions of the models active for each input. As we can observe, removing this functionality from X-Cross results in significant performance drop as well, which attests again to the importance of considering the contributions of other domains.

4.6 Domain Convergence Analysis

We conclude this section by investigating why some domains converge faster than others during training. Referring back to Figure 3, using five random samples of 1000 training examples from each domain's dataset, we observe significant differences in convergence performance, with an average accuracy of 21.82, 19.42, 16.40 and 14.87 for `Sports`, `Toys`, `Electronics` and `Tools` domains, respectively. Despite all domains being trained on identical sample sizes, the disparities in performance are pronounced. Interestingly, the bottom two domains, `Tools` and `Electronics`, represent extremes in dataset size – with `Tools` being the smallest and `Electronics` the largest. This contradiction prompted us to further investigate which possible factors are influencing domain convergence, such as inherent dataset characteristics or domain-specific complexities.

Our initial attempts to explain these differences using classic features of recommendation datasets, such as the number of unique users and items, dataset density, average interactions per user, or per-item distributions, appear to be inconclusive. Similarly, analyzing the characteristics of pre-trained embeddings of individual items or of the entire history of the user offer no clear explanation. However, we do observe that the embedding cosine similarities within domains are exceptionally high, averaging **0.95**, suggesting significant homogeneity in these domains' representations. This drives us even further to investigate prompt-specific features. More specifically, we focus on two main prompt properties: 1. *prompt length* (measured by mean length) and 2. *prompt diversity* (measured as standard deviation of the length). To this end, we train a simple linear regression model with the dependent parameter as the model's accuracy and the regressors are both *prompt length* and *prompt diversity*. Overall, the two prompt properties explain 86% of the variance of the model's accuracy, demonstrating strong relationship between prompt quality and performance. Moreover, based on the regression coefficients, *prompt length* exhibit negative relationship (-0.65) with model accuracy, whereas *prompt diversity* has a positive one (1.60). This suggests that fine-tuning the model with shorter and more diverse prompts results in better accuracy.

5 Conclusion

We have introduced X-Cross, a novel cross-domain sequential recommendation model that dynamically integrates multiple language models at both the layer and input (sample) levels. X-Cross achieves superior performance compared to state-of-the-art single-domain and cross-domain baselines while remaining highly efficient, outperforming parameter-efficient fine-tuning (PEFT) methods like LoRA, particularly in limited training scenarios. Moreover, X-Cross introduces a new approach to integrating LoRA adapters across domains, yielding improvements in cross-domain sequential recommendation tasks that highlight its adaptability and robustness.

As future work, we wish to explore the relationship between source and target domains to better understand how domain selection impacts performance, paving the way for further improvements in cross-domain recommendation systems.

References

[1] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1007–1014.

[2] Tesfaye Fenta Boka, Zhendong Niu, and Rama Bastola Neupane. 2024. A survey of sequential recommendation systems: Techniques, evaluation, and future directions. *Information Systems* 125 (2024), 102427. https://doi.org/10.1016/j.is.2024.102427

[3] Eric L Buehler and Markus J Buehler. 2024. X-LoRA: Mixture of low-rank adapter experts, a flexible framework for large language models with applications in protein mechanics and molecular design. *APL Machine Learning* 2, 2 (2024).

[4] Jin Chen, Zheng Liu, Xu Huang, Chenwang Wu, Qi Liu, Gangwei Jiang, Yuanhao Pu, Yuxuan Lei, Xiaolong Chen, Xingmei Wang, et al. 2024. When large language models meet personalization: Perspectives of challenges and opportunities. *World Wide Web* 27, 4 (2024), 42.

[5] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2024. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems* 36 (2024).

[6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 4171–4186.

[7] Hao Ding, Yifei Ma, Anoop Deoras, Yuyang Wang, and Hao Wang. 2021. Zero-shot recommender systems. *arXiv preprint arXiv:2105.08318* (2021).

[8] Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. 2023. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence* 5, 3 (2023), 220–235.

[9] William Fedus, Jeff Dean, and Barret Zoph. 2022. A review of sparse expert models in deep learning. *arXiv preprint arXiv:2209.01667* (2022).

[10] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

[11] Wenfeng Feng, Chuzhan Hao, Yuewei Zhang, Yu Han, and Hao Wang. 2024. Mixture-of-loras: An efficient multitask tuning for large language models. *arXiv preprint arXiv:2403.03432* (2024).

[12] Junchen Fu, Fajie Yuan, Yu Song, Zheng Yuan, Mingyue Cheng, Shenghui Cheng, Jiaqi Zhang, Jie Wang, and Yunzhu Pan. 2024. Exploring adapter-based transfer learning for recommender systems: Empirical studies and practical insights. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 208–217.

[13] Zihao Fu, Haoran Yang, Anthony Man-Cho So, Wai Lam, Lidong Bing, and Nigel Collier. 2023. On the effectiveness of parameter-efficient fine-tuning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 12799–12807.

[14] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2023. Recommendation as Language Processing (RLP): A Unified Pretrain, Personalized Prompt & Predict Paradigm (P5). arXiv:2203.13366 [cs.IR] https://arxiv.org/abs/2203.13366

[15] Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2021. DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. arXiv:2111.09543 [cs.CL]

[16] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2021. DEBERTA: DECODING-ENHANCED BERT WITH DISENTANGLED ATTENTION. In *International Conference on Learning Representations*. https://openreview.net/forum?id=XPZIaotutsD

[17] Ruining He, Wang-Cheng Kang, and Julian McAuley. 2017. Translation-based recommendation. In *Proceedings of the eleventh ACM conference on recommender systems*. 161–169.

[18] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415* (2016).

[19] Yupeng Hou, Zhankui He, Julian McAuley, and Wayne Xin Zhao. 2023. Learning vector-quantized item representation for transferable sequential recommenders. In *Proceedings of the ACM Web Conference 2023*. 1162–1171.

[20] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. 2024. Bridging language and items for retrieval and recommendation. *arXiv preprint arXiv:2403.03952* (2024).

[21] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards universal sequence representation learning for recommender systems. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 585–593.

[22] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=nZeVKeeFYf9

[23] Damjan Kalajdzievski. 2023. A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA. *arXiv e-prints* (2023), arXiv–2312.

[24] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.

[25] Wang-Cheng Kang, Jianmo Ni, Nikhil Mehta, Maheswaran Sathiamoorthy, Lichan Hong, Ed Chi, and Derek Zhiyuan Cheng. 2023. Do llms understand user preferences? evaluating llms on user rating prediction. *arXiv preprint arXiv:2305.06474* (2023).

[26] Sein Kim, Hongseok Kang, Seungyoon Choi, Donghyun Kim, Minchul Yang, and Chanyoung Park. 2024. Large language models meet collaborative filtering: An efficient all-round llm-based recommender system. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1395–1406.

[27] Jiacheng Li, Ming Wang, Jin Li, Jinmiao Fu, Xin Shen, Jingbo Shang, and Julian McAuley. 2023. Text is all you need: Learning language representations for sequential recommendation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1258–1267.

[28] Jiacheng Li, Yujie Wang, and Julian McAuley. 2020. Time interval aware self-attention for sequential recommendation. In *Proceedings of the 13th international conference on web search and data mining*. 322–330.

[29] Defu Lian, Qi Liu, and Enhong Chen. 2020. Personalized ranking with importance sampling. In *Proceedings of The Web Conference 2020*. 1093–1103.

[30] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Hao Zhang, Yong Liu, Chuhan Wu, Xiangyang Li, Chenxu Zhu, Huifeng Guo, Yong Yu, Ruiming Tang, and Weinan Zhang. 2024. How Can Recommender Systems Benefit from Large Language Models: A Survey. *ACM Trans. Inf. Syst.* (July 2024). https://doi.org/10.1145/3678004

[31] Dairui Liu, Boming Yang, Honghui Du, Derek Greene, Neil Hurley, Aonghus Lawlor, Ruihai Dong, and Irene Li. 2024. RecPrompt: A Self-tuning Prompting Framework for News Recommendation Using Large Language Models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24)*. ACM, 3902–3906. https://doi.org/10.1145/3627673.3679987

[32] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2024. Visual instruction tuning. *Advances in neural information processing systems* 36 (2024).

[33] Peng Liu, Lemei Zhang, and Jon Atle Gulla. 2023. Pre-train, Prompt, and Recommendation: A Comprehensive Survey of Language Modeling Paradigm Adaptations in Recommender Systems. *Transactions of the Association for Computational Linguistics* 11 (2023), 1553–1571.

[34] Shih-yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. 2024. DoRA: Weight-Decomposed Low-Rank Adaptation. In *Forty-first International Conference on Machine Learning*.

[35] I Loshchilov. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).

[36] Sichun Luo, Bowei He, Haohan Zhao, Wei Shao, Yanlin Qi, Yinya Huang, Aojun Zhou, Yuxuan Yao, Zongpeng Li, Yuanzhang Xiao, Mingjie Zhan, and Linqi Song. 2024. RecRanker: Instruction Tuning Large Language Model as Ranker for Top-k Recommendation. arXiv:2312.16018 [cs.IR] https://arxiv.org/abs/2312.16018

[37] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. 2015. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. 43–52.

[38] Akshara Prabhakar, Yuanzhi Li, Karthik Narasimhan, Sham Kakade, Eran Malach, and Samy Jelassi. 2024. LoRA Soups: Merging LoRAs for Practical Skill Composition Tasks. *arXiv e-prints* (2024), arXiv–2410.

[39] Xubin Ren, Wei Wei, Lianghao Xia, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2024. Representation learning with large language models for recommendation. In *Proceedings of the ACM on Web Conference 2024*. 3464–3475.

[40] Uriel Singer, Haggai Roitman, Yotam Eshel, Alexander Nus, Ido Guy, Or Levi, Idan Hasson, and Eliyahu Kiperwasser. 2022. Sequential Modeling with Multiple Attributes for Watchlist Recommendation in E-Commerce. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining* (Virtual Event, AZ, USA) *(WSDM '22)*. Association for Computing Machinery, New York, NY, USA, 937–946. https://doi.org/10.1145/3488560.3498453

[41] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.

[42] Zuoli Tang, Zhaoxin Huan, Zihao Li, Xiaolu Zhang, Jun Hu, Chilin Fu, Jun Zhou, and Chenliang Li. 2023. One model for all: Large language models are domain-agnostic recommendation systems. *arXiv preprint arXiv:2310.14304* (2023).

[43] Ian Tenney, Dipanjan Das, and Ellie Pavlick. 2019. BERT Rediscovers the Classical NLP Pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 4593–4601.

[44] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *Proceedings of the 42nd international ACM SIGIR conference on Research and development in Information Retrieval*. 165–174.

[45] Liwei Wu, Shuqing Li, Cho-Jui Hsieh, and James Sharpnack. 2020. SSE-PT: Sequential recommendation via personalized transformer. In *Proceedings of the*

14th ACM conference on recommender systems. 328–337.

[46] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2024. A survey on large language models for recommendation. *World Wide Web* 27, 5 (2024), 60.

[47] Jingwei Xu, Junyu Lai, and Yunpeng Huang. 2024. MeteoRA: Multiple-tasks Embedded LoRA for Large Language Models. *arXiv e-prints* (2024), arXiv–2405.

[48] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. 2023. Resolving interference when merging models. *arXiv preprint arXiv:2306.01708* 1 (2023).

[49] Prateek Yadav, Derek Tam, Leshem Choshen, Colin A Raffel, and Mohit Bansal. 2024. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems* 36 (2024).

[50] Fajie Yuan, Xiangnan He, Alexandros Karatzoglou, and Liguang Zhang. 2020. Parameter-efficient transfer from sequential behaviors for user modeling and recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 1469–1478.

[51] Fajie Yuan, Guoxiao Zhang, Alexandros Karatzoglou, Joemon Jose, Beibei Kong, and Yudong Li. 2021. One person, one model, one world: Learning continual user representation without forgetting. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 696–705.

[52] Zhenrui Yue, Sara Rabhi, Gabriel de Souza Pereira Moreira, Dong Wang, and Even Oldridge. 2023. LlamaRec: Two-Stage Recommendation using Large Language Models for Ranking. arXiv:2311.02089 [cs.IR] https://arxiv.org/abs/2311.02089

[53] Tianzi Zang, Yanmin Zhu, Haobing Liu, Ruohan Zhang, and Jiadi Yu. 2022. A survey on cross-domain recommendation: taxonomies, methods, and future directions. *ACM Transactions on Information Systems* 41, 2 (2022), 1–39.

[54] Junjie Zhang, Ruobing Xie, Yupeng Hou, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Recommendation as Instruction Following: A Large Language Model Empowered Recommendation Approach. arXiv:2305.07001 [cs.IR] https://arxiv.org/abs/2305.07001

[55] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. 2023. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 3836–3847.

[56] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S Sheng, Jiajie Xu, Deqing Wang, Guanfeng Liu, Xiaofang Zhou, et al. 2019. Feature-level deeper self-attention network for sequential recommendation.. In *IJCAI*. 4320–4326.

[57] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).

[58] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 1893–1902.

[59] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. 2022. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems* 35 (2022), 7103–7114.